Rajarata University of Sri Lanka
Department of Computing

ICT1402
PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 05

Introduction to C Programming

K.A.S.H. Kulathilake
Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

- At the end of this lecture students should be able to;
  - Describe features of C programming language.
  - Justify the terminology related to computer programming.
  - Define the editing, compiling, linking, debugging stages of C programming
  - Recognize the basic structure of a C program
  - Apply comments for C programs to improve readability.

History

- The C programming Language was pioneered by Dennis Ritchie at AT&T Bell Laboratories in the early 1970s.
- C became popular with the development of UNIX operating system which was developed in the same laboratory.
- C grew in popularity across different operating systems as a result of marketing different C compilers produced by different vendors.
- In 1990, the first official ANSI standard definition of C was published.
- In 1999, ISO standard for C was published.

Programming

- Computers are really very dumb machines indeed because **they do only what they are told to do**.
- The basic operations of a computer system form what is known as the ***computer's instruction set***.
- To solve a problem using a computer, you must express the solution to the problem in terms of the instructions of the particular computer.
- **A computer program** is just a collection of instructions necessary to solve a specific problem.
- The approach or method that is used to solve the problem is known as an **algorithm**.

Programming (Cont…)

Normally, to develop a program to solve a particular problem, you first express the solution to the problem in terms of an algorithm and then develop a program that implements that algorithm.

High Level Languages

Programs written in assembly language are not portable

```
swap:
        muli    $t0, $a0, 4
        add     $t0, $a1, $t0
        lw      $t1, 0($t0)
        lw      $t2, 4($t0)
        sw      $t2, 0($t0)
        sw      $t1, 4($t0)
        jr      $ra
```

compiler

```
void swap(int v[], int k)
{
        int temp;
        temp = v[k];
        v[k] = v[k+1];
        v[k+1] = temp;
}
```

assembler

```
00000000101000010…
00000000000110000…
10001100011000100…
10001100111100100…
10101100111100100…
10101100011000100…
00000011111000000…
```

In Assembly Language. Programmer used symbolic names to represents the operations

Assembler translates the assembly codes to machine instructions

Have 1-to1 correspondent between assembly language and machine language.

High Level Languages (Cont…)

What is meant by "the programs written in Low Level Languages are not Portable"?

That is, the program will not run on a different processor type without being rewritten. This is because different processor types have different instruction sets, and because assembly language programs are written in terms of these instruction sets, they are machine dependent.

High Level Languages (Cont…)

- Programmers developing programs in high level languages no longer had to concern themselves with the architecture of the particular computer.
- Operations performed in High level languages were of a much more sophisticated or higher level, far removed from the instruction set of the particular machine.
- One High Level Language instruction or statement resulted in many different machine instructions being executed, unlike the one-to-one correspondence found between assembly language statements and machine instructions.

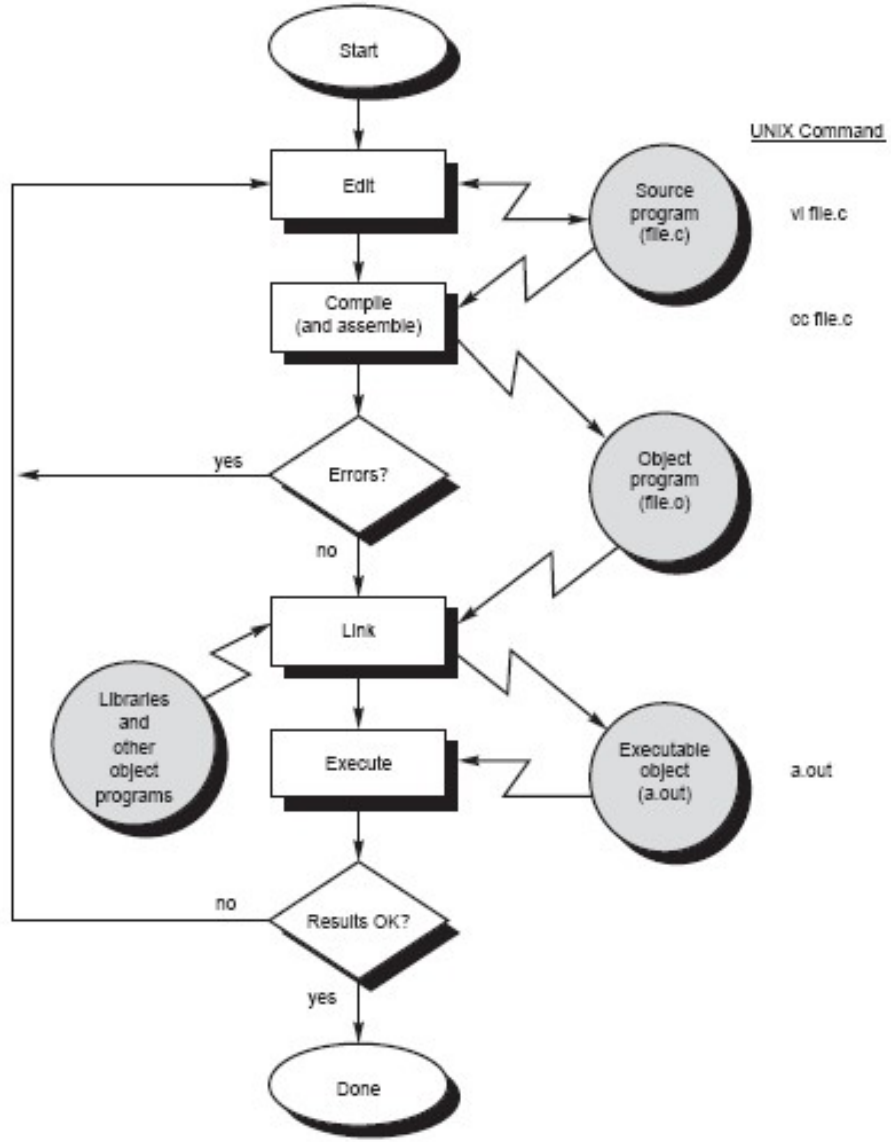
High Level Languages (Cont…)

- Standardization of the syntax of a Higher-Level Language meant that a program could be written in the language to be machine independent.
- That is, a program could run on any machine that supported the language with few or no changes.

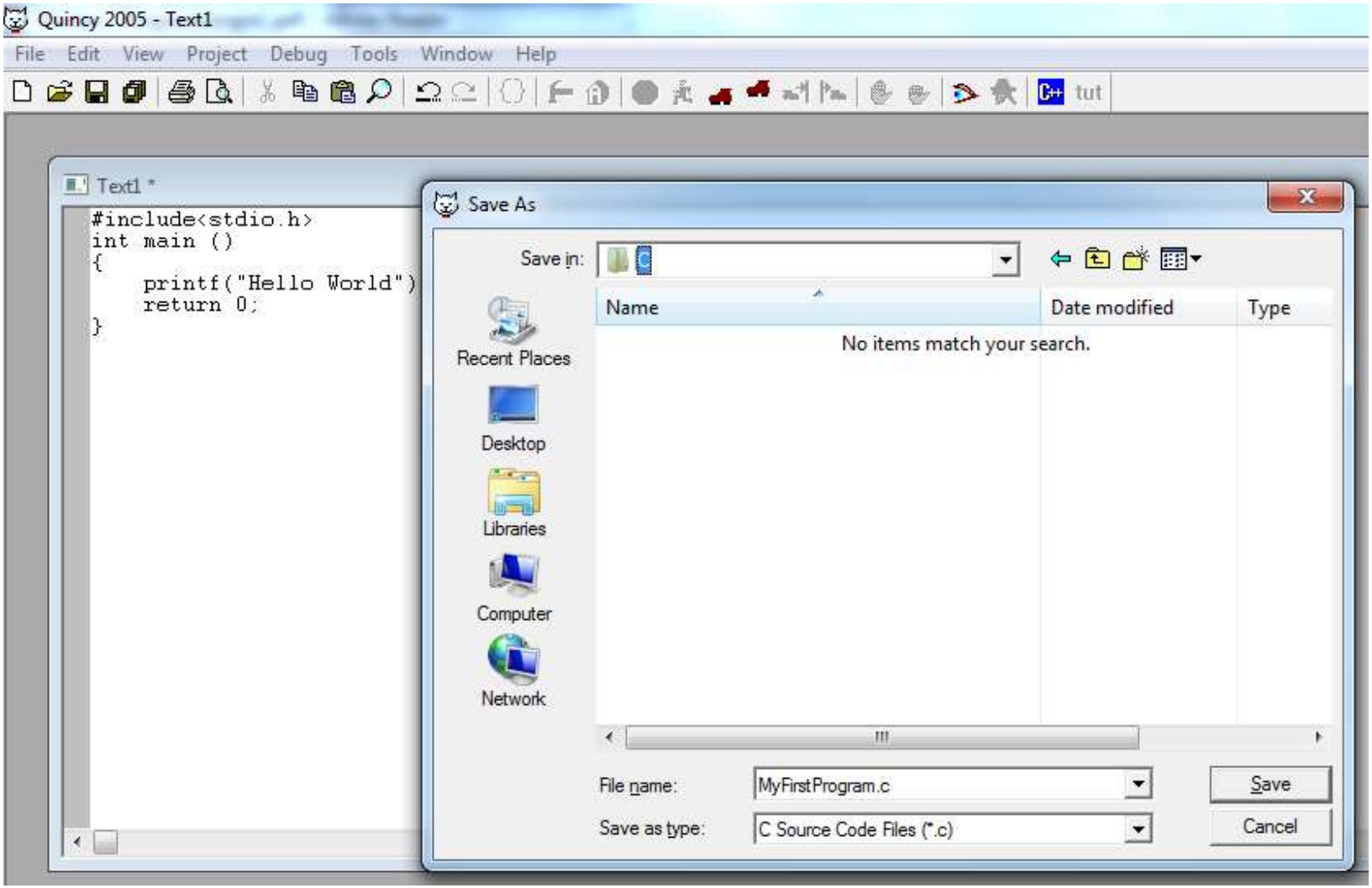
To support a higher-level language, a special computer program must be developed that translates the statements of the program developed in the higher-level language into a form that the computer can understand—in other words, into the particular instructions of the computer. Such a program is known as a *COMPILER*.

Compiling Programs

- Editing
  - First write your program using text editor or Integrated Development Environment (IDE).
  - Then you need to save your file by giving a suitable name with ".c" file extension.
    - e.g.
    - MyFirstProgram.c



Compiling Programs (Cont…)



Compiling Programs (Cont…)

- Compiling
  - The program that is entered into the file is known as the ***source program*** because it represents the original form of the program expressed in the C language.
  - Then you can compile your program using c compiler;
  - If you are using the popular GNU C compiler in UNIX, the command you use is gcc.
  - Type following command to compile your program

    gcc MyFirstProgram.c

Compiling Programs (Cont…)

- In the first step of the compilation process, the compiler examines each program statement contained in the source program and checks it to ensure that it conforms to the syntax of the language.
- During the compilation process, compiler scans the source file and checks for the syntactic errors in the source file.
  - If passed, you can proceed ahead
  - If failed, compiler indicates the errors and you have to fix them and re-compile your source file after saving the changes.

Compiling Programs (Cont…)

- When all the syntactic and semantic errors have been removed from the program, the compiler then proceeds to take each statement of the program and translate it into a "lower" form.
- On most systems, this means that each statement is translated by the compiler into the equivalent statement or statements in assembly language needed to perform the identical task.
- After the program has been translated into an equivalent assembly language program, the next step in the compilation process is to translate the assembly language statements into actual machine instructions.

Compiling Programs (Cont…)

- This step might or might not involve the execution of a separate program known as an ***assembler***.
- On most systems, the assembler is executed automatically as part of the compilation process.
- **The assembler takes each assembly language statement and converts it into a binary format known as object code**, which is then written into another file on the system.
- This file typically has the same name as the source file under Unix, with the last letter an “o” (for object) instead of a “c”.
- Under Windows, the suffix letters "obj" typically replace the “c” in the filename.

Compiling Programs (Cont…)

- Linking
  - After the program has been translated into object code, it is ready to be *linked*.
  - This process is once again performed automatically whenever the cc or gcc command is issued under Unix.
  - **The purpose of the linking phase is to get the program into a final form for execution on the computer.**
  - If the program uses other programs that were previously processed by the compiler, then during this phase those programs are linked together.
  - Programs that are used from the system's program library are also searched and linked together with the object program during this phase.

Compiling Programs (Cont…)

- **The process of compiling and linking a program is often called *building*.**
- The final linked file, which is in an **executable object code** format, is stored in another file on the system, ready to be run or executed.
- Under Unix, this file is called a.out by default.
- Under Windows, the executable file usually has the same name as the source file, with the c extension replaced by an exe extension.
- So, the command;
  - MyFirstProgram.out in UNIX
  - MyFirstProgram.exe in Windows

  has the effect of loading the program called MyFirstProgram.out/ MyFirstProgram.exe into the computer's memory and initiating its execution.

Compiling Programs (Cont…)

- When the program is executed, each of the statements of the program is sequentially executed in turn.
- If the program requests any data from the user, known as *input, the* program temporarily suspends its execution so that the input can be entered.
- Or, the program might simply wait for an event, such as a mouse being clicked, to occur.
- Results that are displayed by the program, known as output, appear in a window, sometimes called the console.
- Or, the output might be directly written to a file on the system.

Compiling Programs (Cont…)

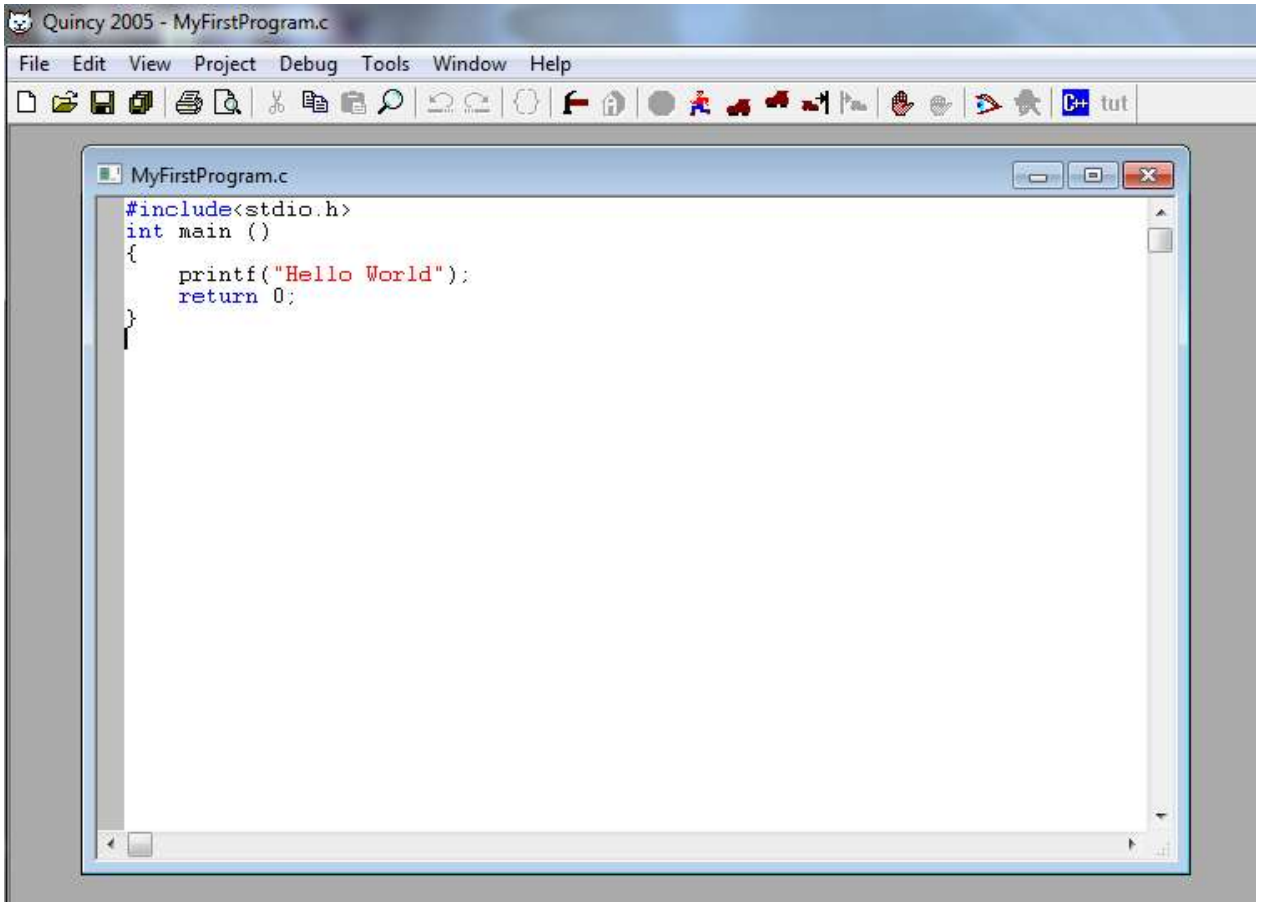
- Debugging
  - If all goes well (and it probably won't the first time the program is executed), the program performs its intended functions.
  - If the program does not produce the desired results, it is necessary to go back and reanalyze the program's logic.
  - This is known as the ***debugging phase***, during which an attempt is made to remove all the known problems or bugs from the program.
  - To do this, it will most likely be necessary to make changes to the original source program.
  - In that case, the entire process of compiling, linking, and executing the program must be repeated until the desired results are obtained.

IDE

- The process of editing, compiling, running, and debugging programs is often managed by a single integrated application known as an Integrated Development Environment, or IDE for short.
- An IDE is a windows-based program that allows you to easily manage large software programs, edit files in windows, and compile, link, run, and debug your programs.

Introduction to Quincy C

- http://www.codecutter.net/tools/quincy/
- Refer Note



C Program Structure

```
/* Header Filed Placed Here*/
/* Global Data Placed Here */

function 1 () {
    /* Local Data and  C code placed here */
}

Function n () {
    /* Local Data and  C code placed here */
}

main () { /* Entry Point */
    /* Local Data and  C code placed here */
}
```

Depicts a typical single source file C program.

Every C Program consists of 1 or more functions.

main() is the mandatory function in every C program.

First C Program

```
#include<stdio.h>
int main ()
{
    printf("Programming is Fun\n");
    return 0;
}
```

In the C programming language, lowercase and uppercase letters are distinct.

CASE SENSITIVE

First C Program (Cont…)

- Compile and Run

MyFirstProgram.c

```c
#include<stdio.h>
int main ()
{
    printf("Programming is Fun\n");
    return 0;
}
```

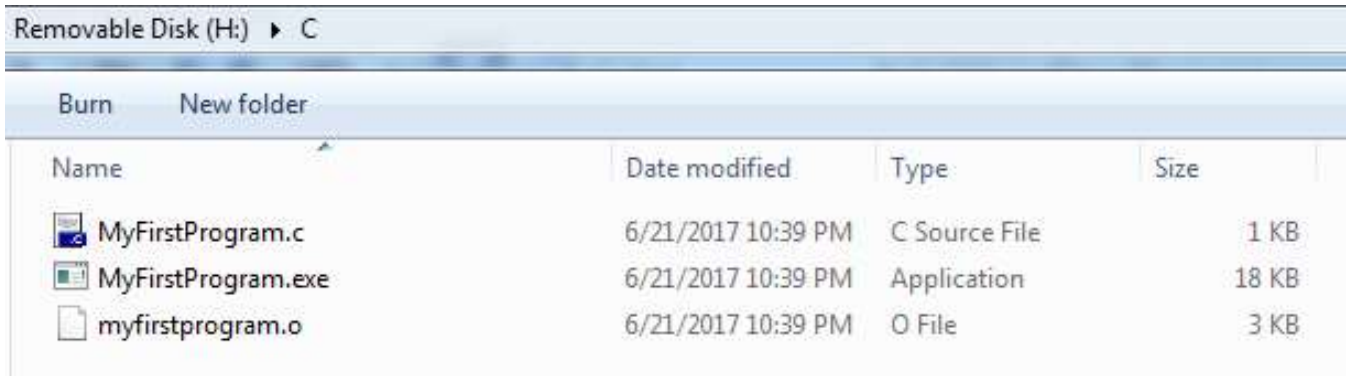
C:\Windows\system32\cmd.exe

```
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

H:\C>gcc MyFirstProgram.c

H:\C>MyFirstProgram.exe
Programming is Fun

H:\C>^C
```

Try : gcc MyFirstProgram.c –o MyFirstProgram.exe

First C Program (Cont…)

- Contents in the Saved Location



First C Program (Cont…)

```
#include<stdio.h>
```

- Should be included at the beginning of just about every program you write.
- It tells the compiler to include the standard input output header file stdio.h as part of the program.
- It contains the information about the printf output routine that is used later in the program.

First C Program (Cont…)

```
int main (void)
```

- Informs the system that the name of the program is main, and that it returns an integer value, which is abbreviated “int.”
- **Main is a special name that indicates precisely where the program is to begin execution.**
- The open and close parentheses immediately following main specify that main is the name of a function.
- The keyword void that is enclosed in the parentheses specifies that the function main takes no arguments (that is, it is void of arguments).

First C Program (Cont…)

{ }

- Describe the boundary of the main function.
- This is done by enclosing all program statements of the routine within a pair of curly braces.
- All program statements included between the braces are taken as part of the main routine by the system.
- You have only two statements enclosed within the braces of this program.

First C Program (Cont…)

```
printf("Programming is Fun\n");
```

- The first statement specifies that a routine named printf is to be invoked or called.
- The parameter or argument to be passed to the printf routine is the string of characters "Programming is fun.\n"
- The printf routine is a function in the C library that simply prints or displays its argument (or arguments, as you will see shortly) on your screen.
- \n is the new line character

First C Program (Cont…)

`\n`

- Any characters to be printed after the newline character then appear on the next line of the display.
- In fact, the newline character is similar in concept to the carriage return key on a typewriter.
- ** All program statements in C must be terminated by a semicolon (;).

First C Program (Cont…)

```
return 0
```

- says to finish execution of main, and return to the system a status value of 0.
- You can use any integer here.
- Zero is used by convention to indicate that the program completed successfully—that is, without running into any errors.

First C Program (Cont…)

What are Header files ?

C provides a collection of useful functions which are called as library functions.
A library is split into a group of functions and each group has a .h (header) file associated with it.
Often the .h file will contain other components such as type declarations.
Whenever the functions in a given group are used, the group's .h file should be included with #include<>.

Adding another Phrase

```
#include <stdio.h>
int main (void)
{
    printf ("Programming is fun.\n");
    printf ("And programming in C is
             even more fun.\n");
    return 0;
}
```

Using \n

```
#include <stdio.h>
int main (void)
{
    printf
  ("Testing...\n..1\n...2\n....3\n");
    return 0;
}
```

Comments

- A comment statement is used in a program to document a program and to enhance its readability.
- comments serve to tell the reader of the program;
  - The programmer or someone else whose responsibility it is to maintain the program
  - Just what the programmer had in mind when he or she wrote a particular program or a particular sequence of statements.

Comments (Cont…)

```
/* This program demonstrate the application */
/* of new line character */

#include <stdio.h>

int main (void)
{
    /* Display text with new line character*/
    printf ("Testing...\n..1\n...2\n....3\n");
    return 0;
}
```

Comments (Cont…)

- Comments are ignored by the compiler.
- There are two ways to insert comments;
  - /* */ - This form of comment is often used when comments span several lines in the program.
  - // - Any characters that follow these slashes up to the end of the line are ignored by the compiler.

Comments (Cont…)

- It is a good idea to get into the habit of inserting comment statements into the program as the program is being written or typed in.
- There are good reasons for this;
  - It is far easier to document the program while the particular program logic is still fresh in your mind than it is to go back and rethink the logic after the program has been completed.
  - By inserting comments into the program at such an early stage of the game, you get to reap the benefits of the comments during the debug phase, when program logic errors are being isolated and debugged.
- A comment can not only help you read through the program, but it can also help point the way to the source of the logic mistake.

Objective Re-cap

- Now you should be able to:
  - Describe features of C programming language.
  - Justify the terminology related to computer programming.
  - Define the editing, compiling, linking, debugging stages of C programming
  - Recognize the basic structure of a C program
  - Apply comments for C programs to improve readability.

References

- Chapter 01,02 and 03 - Programming in C, 3rd Edition, Stephen G. Kochan

Q & A

Next: Variables and Data Types